Bài 4 — Làm quen với pandas: 60 câu hỏi

- Giả định pandas 3.0.x với Copy-on-Write; notebook tham chiếu 3.0.3. Code đã kiểm chứng trên pandas 3.0.1.
- Giả định `import pandas as pd` và `import numpy as np` đã chạy. Mỗi câu độc lập; khi chuyển câu, khởi tạo lại dữ liệu của câu đó.
- Cột giá trong các ví dụ số/NaN là `float64`; NaN là `np.nan`, không dùng nullable Float64/pd.NA.
- Cảnh báo được hiển thị và không cấu hình warnings thành errors. `ChainedAssignmentError` trong các câu này là cảnh báo, không tự làm dừng chương trình.
- Mỗi câu trắc nghiệm có đúng một phương án A/B/C/D. Ký hiệu `nan` trong phương án là giá trị thiếu. Bỏ qua khác biệt khoảng trắng khi in danh sách/tuple.
- Các bảng là giả lập; mã NC/PR/Shared và các tên loại được hiểu đúng như ghi trong từng câu. Phí ghi "một lần" chỉ thu một lần cho một lượt ở.
- Không yêu cầu nhớ số liệu snapshot Santiago hoặc số byte của môi trường khác. Câu 050 cung cấp số đo quan sát để diễn giải.

Bộ luyện tập gồm 40 câu trắc nghiệm và 20 câu tự luận ngắn, chia sáu đợt.

Đợt 1 — B4-001 đến B4-010

7 câu đầu trắc nghiệm; 3 câu cuối tự luận.

B4-001

```
df = pd.DataFrame({"ma": [11, 12, 13], "gia": [60., np.nan, 90.]},
                  index=["k", "m", "p"])
print(df.shape, df.columns.tolist())
```

Dòng cuối in ra gì?

A. `(3, 3) ['ma', 'gia']`

B. `(2, 3) ['ma', 'gia']`

C. `(3, 2) ['ma', 'gia']`

D. `(3, 2) ['k', 'm', 'p']`

B4-002

```
df = pd.DataFrame({"gia": [40., np.nan, 0.], "phi": [5., 5., 0.]},
                  index=["r3", "r1", "r8"], dtype="float64")
t = df["gia"] * 2
print(t.name, t.index.tolist(), t.tolist(), df.loc["r3", "gia"])
```

Dòng cuối in ra gì? `nan` trong các phương án biểu thị giá trị thiếu.

A. `gia ['r3', 'r1', 'r8'] [80.0, nan, 0.0] 40.0`
B. `gia [0, 1, 2] [80.0, nan, 0.0] 40.0`
C. `gia ['r3', 'r1', 'r8'] [80.0, 0.0, 0.0] 40.0`
D. `gia ['r3', 'r1', 'r8'] [80.0, nan, 0.0] 80.0`

B4-003

Tệp `cho_o.csv` có nội dung:

id	loai	price
101	NC	50000
102	PR	30000
103	NC	70000

Mô tả nguồn quy định mỗi hàng là một chỗ ở; `price` tính bằng CLP/đêm.

```
df = pd.read_csv("cho_o.csv")
print(df["id"].mean())
```

Dòng cuối in `102.0` . Nhận định nào đúng?

A. `df` là DataFrame gồm ba lượt đặt phòng; `102.0` là số lượt đặt trung bình.
B. `df` là DataFrame gồm ba chỗ ở; `id` là mã, `loai` là nhãn phân loại, `price` là số đo CLP/đêm; `102.0` không phải giá trung bình.
C. `df` là Series gồm ba chỗ ở; `id` là mã, `loai` là nhãn phân loại, `price` là số đo CLP/đêm.
D. `df` là DataFrame gồm ba chỗ ở; `102.0` là mức giá mỗi đêm điển hình, tính bằng CLP.

B4-004

Theo quy ước nguồn, `0.0` là giá miễn phí hợp lệ; `np.nan` là chưa biết giá.

```
s = pd.Series([0., np.nan, 50., 100.], dtype="float64")
print(len(s), s.count(), (s < 60).mean())
```

Dòng cuối in ra gì?

A. `4 4 0.5`

B. `4 3 0.6666666666666666`

C. `4 3 0.5`

D. `4 3 0.25`

B4-005

```
df = pd.DataFrame({"id": ["A", "B", "C", "D", "E", "F"],
                   "gia": [140., 30., 70., 110., 50., 90.]})
r = df[df["gia"] < 100]
print(r.loc[2:4, "gia"].sum(), r.iloc[2:4, 1].sum())
```

Dòng cuối in ra gì?

A. `210.0 140.0`

B. `120.0 120.0`

C. `140.0 120.0`

D. `120.0 140.0`

B4-006

```
df = pd.DataFrame({"gia": [80., 40., 60.]},
                  index=["u", "v", "w"], dtype="float64")
df["hai_dem"] = df["gia"] * 2
a = df
b = df.copy()
a.loc["u", "gia"] = 100.
b.loc["u", "gia"] = 5.
print(df.loc["u", ["gia", "hai_dem"]].tolist(),
      b.loc["u", ["gia", "hai_dem"]].tolist())
```

Lệnh print in ra gì?

A. `[5.0, 160.0] [5.0, 160.0]`
B. `[100.0, 160.0] [5.0, 160.0]`
C. `[100.0, 200.0] [5.0, 10.0]`
D. `[80.0, 160.0] [5.0, 160.0]`

B4-007

```
df = pd.DataFrame({"gia": [20., 40., 60.], "phi": [0., 5., 10.]},
                  index=["N1", "N2", "N3"], dtype="float64")
df.index.name = "ma"
df[["gia"]].to_csv("bang.csv", index=False)
lai = pd.read_csv("bang.csv")
print(lai.shape, lai.columns.tolist())
```

Nhận định nào đúng?

A. In `(3, 1) ['gia']` ; file không giữ các mã N1, N2, N3.
B. In `(3, 2) ['ma', 'gia']` ; index có tên nên các mã vẫn được ghi.
C. In `(3, 2) ['Unnamed: 0', 'gia']` ; index được ghi thành cột phụ.
D. `read_csv` báo lỗi vì file không có cột chỉ mục.

B4-008

Mỗi hàng là một chỗ ở. Giá 0 hợp lệ; NaN là chưa biết.

```
df = pd.DataFrame({
    "id": ["A1", "A2", "A3", "A4", "A5"],
    "gia": pd.Series([0., 40., np.nan, 120., np.nan], dtype="float64")
})
p = (df["gia"] < 80).mean()
print(p)
```

Một bạn nói: “Có 40% chỗ ở giá dưới 80, nên 60% chỗ ở có giá từ 80 trở lên.”

a) Nêu mẫu số của p và giải thích kết luận “60%” có đúng không. Có nên đổi hai giá thiếu thành 0 để sửa phép tính không?

b) Viết một hoặc hai dòng tính q: tỷ lệ giá dưới 80 trong các chỗ ở đã có giá. Nêu q.

c) Đổi p, q thành phần trăm và tính chênh lệch bằng điểm phần trăm.

B4-009

Yêu cầu: tạo báo giá ba đêm, giảm 25% giá loại NC, giữ giá PR và giữ nguyên df.

```
df = pd.DataFrame({"loai": ["NC", "PR", "NC"],
                   "gia": [120., 50., 80.]}, index=["x", "y", "z"])
bao_gia = df.copy()
bao_gia["ba_dem"] = bao_gia["gia"] * 3
mask = bao_gia["loai"] == "NC"
s = bao_gia.loc[mask, "gia"]
s *= 0.75
print(bao_gia.loc["x", ["gia", "ba_dem"]].tolist())
```

a) Dòng cuối in gì? Vì sao giảm trên s chưa đáp ứng yêu cầu?

b) Thay dòng `s *= 0.75` bằng một lệnh gán đúng vào bảng cần sửa.

c) Ngay sau khi chỉ sửa b), dòng cuối in gì? Đã đủ yêu cầu chưa? Viết phần còn thiếu và nêu kết quả cần kiểm ở bao_gia và df.

B4-010

```
df = pd.DataFrame({"id": ["Q1", "Q2", "Q3", "Q4"],
                   "gia": [70., 150., 40., 100.]}, index=[10, 20, 30, 40])
r = df[df["gia"] <= 100]
print(r.loc[1, "gia"])
```

Người viết muốn lấy giá của hàng thứ hai trong kết quả lọc.

a) Dòng cuối có thực hiện được yêu cầu không? Nêu các nhãn hàng của r và giải thích.

b) Sửa dòng cuối để lấy đúng giá cần tìm. Cho biết giá đó.

c) Nêu kiểu và shape của `r.loc[[30], "gia"]` và `r.loc[[30], ["gia"]]` .

Đợt 2 — B4-011 đến B4-020

7 câu đầu trắc nghiệm; 3 câu cuối tự luận.

B4-011

```
df = pd.DataFrame({
    "loai": ["PR", "NC", "PR"],
    "gia": [45., np.nan, 75.]
})
u = df["gia"]
v = df[["gia"]]
print(type(u).__name__, u.shape, u.dtype)
print(type(v).__name__, v.shape)
```

Hai dòng cuối in ra gì?

Phương án	Dòng thứ nhất	Dòng thứ hai
A	`Series (3, 1) float64`	`DataFrame (3,)`
B	`DataFrame (3, 1) float64`	`Series (3,)`
C	`Series (3,) int64`	`DataFrame (3, 1)`
D	`Series (3,) float64`	`DataFrame (3, 1)`

B4-012

```
df = pd.DataFrame({
    "id": [11, 12, 13, 14, 15, 16],
    "loai": ["NC", "PR", "NC", "PR", "NC", "PR"],
    "gia": [20., np.nan, 40., 60., np.nan, 80.]
})
df.info()
print(df["gia"].describe())
```

Mỗi hàng là một chỗ ở. Trích các dòng cần thiết trong output:

```
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
id      6 non-null  int64
loai    6 non-null  str
gia     4 non-null  float64

count     4.0
mean     50.0
min      20.0
50%      50.0
max      80.0
```

Nhận định nào đúng?

A. Thiếu 2 giá; mean 50 được tính trên cả 6 hàng, với giá thiếu được tính như 0.
B. Thiếu 2 giá; mean 50 được tính trên 4 giá đã biết, không dùng hai ô thiếu làm giá 0.
C. Thiếu 4 giá; mean 50 được tính trên 2 giá đã biết, không dùng bốn ô thiếu làm giá 0.
D. Thiếu 2 giá; dòng 50% cho biết 50% số chỗ ở có giá đúng bằng 50.

B4-013

```python
s = pd.Series([0., np.nan, 30., 60.], dtype="float64")
print((s == np.nan).sum(), s.isna().sum(), s.count())
```

Dòng cuối in ra gì?

A. `0 1 3`
B. `1 1 3`
C. `0 0 4`
D. `0 2 2`

B4-014

```
df = pd.DataFrame({
    "id": ["H1", "H2", "H3", "H4", "H5", "H6"],
    "loai": ["Studio", "Phòng riêng", "Studio",
             "Phòng riêng", "Phòng chung", "Studio"],
    "gia": [90., 60., 100., 75., 40., np.nan]
})
m = (((df["loai"] == "Studio") & (df["gia"] <= 100))
     | ((df["loai"] == "Phòng riêng") & (df["gia"] < 75)))
print(df.loc[m, "id"].tolist())
```

Dòng cuối in ra gì?

A. `['H1', 'H2', 'H3', 'H4']`
B. `['H1', 'H2']`
C. `[]`
D. `['H1', 'H2', 'H3']`

B4-015

```
df = pd.DataFrame({
    "loai": ["A", "B", "A"], "gia": [80., 50., 100.]
})
df["gia"][df["loai"] == "A"] = 60.
df.loc[df["loai"] == "B", "gia"] = 45.
print(df["gia"].tolist())
```

Nhận định nào đúng về việc chạy đoạn code?

A. Có cảnh báo, code chạy tiếp và in `[60.0, 45.0, 60.0]`.
B. Có ngoại lệ `ChainedAssignmentError`, code dừng trước khi chạy `print`.
C. Có cảnh báo, code chạy tiếp và in `[80.0, 45.0, 100.0]`.
D. Có cảnh báo, code chạy tiếp và in `[80.0, 50.0, 100.0]`.

B4-016

```
df = pd.DataFrame({"id": ["M8", "M2", "M5"],
                   "gia": [30., 60., 90.]}, index=[7, 2, 9])
df.index.name = "dong"
df.to_csv("bang_016.csv")
lai = pd.read_csv("bang_016.csv")
print(lai.columns.tolist(), lai["id"].tolist())
```

Dòng cuối in ra gì?

A. `['dong', 'id', 'gia'] ['M8', 'M2', 'M5']`
B. `['id', 'gia'] ['M8', 'M2', 'M5']`
C. `['Unnamed: 0', 'id', 'gia'] ['M8', 'M2', 'M5']`
D. `['dong', 'id', 'gia'] [7, 2, 9]`

B4-017

```
df = pd.DataFrame({
    "id": ["C1", "C2", "C3", "C4", "C5", "C6"],
    "khu": ["Bắc", "Bắc", np.nan, "Nam", "Tây", "Bắc"]
})
print(df["khu"].nunique())
```

Dòng cuối in ra số nào, và cột nào chứa nhãn phân loại lặp lại để cân nhắc dùng dtype category? Không yêu cầu kết luận về số byte tiết kiệm được.

A. `4` ; cột `khu` .
B. `3` ; cột `khu` .
C. `3` ; cột `id` .
D. `6` ; cột `khu` .

B4-018

Giá dưới đây tính bằng USD/đêm của cùng bốn chỗ ở qua hai lần thu thập:

```
df = pd.DataFrame({
    "lan_1": [30., 60., 90., 180.],
    "lan_2": [40., 70., 100., np.nan]
}, index=["a", "b", "c", "d"], dtype="float64")
```

a) Tính trung bình các giá đã biết ở mỗi lần. Nếu chỉ nhìn hai trung bình này, mức trung bình thay đổi bao nhiêu?
b) Viết vài dòng xác định các hàng có giá ở cả hai lần rồi tính hai trung bình trên đúng tập hàng chung đó. Cho biết kết quả.
c) Giải thích vì sao hai cách cho chiều thay đổi khác nhau. Dữ liệu hỗ trợ kết luận gì về ba chỗ ở thuộc nhóm chung? Có suy ra được giá lần 2 của d không?

B4-019

Yêu cầu: trên bản sao bao_gia, giảm 20% giá loại NC, giữ giá loại PR, tính tiền hai đêm theo giá mới và giữ nguyên bảng nguon.

```
nguon = pd.DataFrame({
    "loai": ["NC", "PR", "NC", "PR"],
    "gia": [150., 50., 100., 80.]
}, index=["D1", "D2", "D3", "D4"])
bao_gia = nguon.copy()
bao_gia["hai_dem"] = bao_gia["gia"] * 2
m = bao_gia["loai"] == "NC"
bao_gia[m]["gia"] = bao_gia.loc[m, "gia"] * 0.8
print(bao_gia["gia"].sum(), bao_gia["hai_dem"].sum())
```

a) Dòng cuối in ra gì? Giải thích vì sao phép gán qua bao_gia[m] không đạt yêu cầu.
b) Thay riêng dòng gán giá bằng một lệnh gán đúng, sử dụng mask m đã có.
c) Ngay sau khi chỉ sửa ở b), hai tổng in ra là bao nhiêu? Đã đủ yêu cầu chưa? Viết phần còn thiếu và nêu cách kiểm giá, tiền hai đêm, các hàng PR và bảng nguồn.

B4-020

```
df = pd.DataFrame({
    "id": ["A7", "B2", "C9", "D4", "E1", "F6"],
    "loai": ["Studio", "Phòng riêng", "Phòng chung",
             "Studio", "Phòng riêng", "Phòng chung"],
    "gia": [95., 50., 20., 75., 65., 30.]
}, index=[12, 4, 20, 7, 15, 9])
```

a) Dùng isin để tạo DataFrame r chỉ gồm Studio hoặc Phòng riêng, sắp giá tăng dần, giữ đúng hai cột theo thứ tự gia, id và giữ nguyên nhãn hàng từ df.
b) Viết bảng kết quả, gồm cả nhãn hàng. Không có giá đồng hạng trong dữ liệu này.

Đợt 3 — B4-021 đến B4-030

7 câu đầu trắc nghiệm; 3 câu cuối tự luận.

B4-021

```
df = pd.DataFrame({"loai": ["A", "B", "A"], "gia": [12., np.nan, 18.]},
                  index=["x", "y", "z"])
t = df.dtypes
print(type(t).__name__, t.index.tolist(), str(t["loai"]), str(t["gia"]))
print(isinstance(df.index, pd.Index), isinstance(df.columns, pd.Index))
```

Hai dòng cuối in gì trong pandas 3?

Phương án	Dòng 1	Dòng 2
A	`DataFrame ['loai', 'gia'] str float64`	`True True`
B	`Series ['x', 'y', 'z'] str float64`	`True True`
C	`Series ['loai', 'gia'] object float64`	`True True`
D	`Series ['loai', 'gia'] str float64`	`True True`

B4-022

```
df = pd.DataFrame({"id": ["A", "B", "C", "D", "E"],
                   "gia": [900., 800., 20., 30., 40.]})
dau = df.head(2)
mau = df.sample(2, random_state=9)
```

Chạy lại từ đầu trên cùng dữ liệu, cùng thứ tự hàng và cùng môi trường. Nhận định nào đúng?

A. head chọn ngẫu nhiên nên dau có thể đổi; sample luôn chọn hai giá thấp nhất.
B. head lấy hai hàng đầu; random_state bảo đảm mau đại diện đúng phân bố toàn bảng.
C. head lấy hai hàng đầu; mau có thể lặp lại, nhưng hai cách xem này không bảo đảm tính đại diện.
D. head lấy hai hàng đầu; random_state bảo đảm trung bình mau bằng trung bình toàn bảng.

B4-023

```
s = pd.Series([10., 20., 30., 340., np.nan], dtype="float64")
r = s.agg(["mean", "median", "min", "max", "sum"])
print(r.tolist())
```

Dòng cuối in gì?

A. `[100.0, 25.0, 10.0, 340.0, 400.0]`
B. `[80.0, 20.0, 0.0, 340.0, 400.0]`
C. `[100.0, 30.0, 10.0, 340.0, 400.0]`
D. `[100.0, 25.0, 10.0, 340.0, nan]`

B4-024

```
df = pd.DataFrame({"gia": [20., 50., 80.], "phi": [1., 2., 3.]},
                  index=["p", "q", "r"], dtype="float64")
k = df.loc[["r", "p"], ["phi", "gia"]]
print(type(k).__name__, k.index.tolist(), k.columns.tolist(), k.iloc[0, 1])
```

Dòng cuối in gì?

A. `DataFrame ['p', 'r'] ['phi', 'gia'] 20.0`
B. `DataFrame ['r', 'p'] ['gia', 'phi'] 3.0`

C. `DataFrame ['r', 'p'] ['phi', 'gia'] 80.0`

D. `Series ['r', 'p'] ['phi', 'gia'] 80.0`

B4-025

```
df = pd.DataFrame({"gia": [25., np.nan, 40.], "phi": [5., 10., 0.]},
                  dtype="float64")
df["tong"] = df["gia"] * 2 + df["phi"]
print(df.shape, df["tong"].tolist())
```

Dòng cuối in gì?

A. `(3, 2) [55.0, nan, 80.0]`

B. `(3, 3) [55.0, 10.0, 80.0]`

C. `(3, 3) [60.0, nan, 80.0]`

D. `(3, 3) [55.0, nan, 80.0]`

B4-026

Phí phi thu một lần cho một lượt ở; gia tính theo USD/đêm.

```
df = pd.DataFrame({"gia": [100., 70., 50.], "phi": [0., 80., 200.]},
                  index=["A", "B", "C"], dtype="float64")
df["hai"] = df["gia"] * 2 + df["phi"]
df["tam"] = df["gia"] * 8 + df["phi"]
print(df["hai"].tolist(), df["tam"].tolist())
```

Dòng cuối in gì?

A. `[200.0, 300.0, 500.0] [800.0, 1200.0, 2000.0]`

B. `[200.0, 140.0, 100.0] [800.0, 560.0, 400.0]`

C. `[200.0, 220.0, 300.0] [800.0, 640.0, 600.0]`

D. `[200.0, 220.0, 300.0] [800.0, 560.0, 400.0]`

B4-027

```
goc = pd.DataFrame({"id": ["A", "B", "C"], "gia": [10., 20., 40.]})
gui = pd.DataFrame({"id": ["A", "B", "C"], "gia": [40., 20., 10.]})
gui.to_csv("bang_027.csv", index=False)
lai = pd.read_csv("bang_027.csv")
print(lai.shape == goc.shape, lai["gia"].sum() == goc["gia"].sum())
```

Nhận định nào đúng?

A. In `True True` ; vì vậy cả ba cặp id–giá đều khớp goc.
B. In `True True` ; giá của A và C vẫn khác goc.
C. In `True False` ; tổng đã phát hiện việc hoán đổi giá.
D. In `False True` ; số hàng đã phát hiện việc hoán đổi giá.

B4-028

```
df = pd.DataFrame({"id": ["X1", "X2", "X3", "X4", "X5"],
                   "gia": [20., 30., 40., 50., 860.]}, index=[5, 9, 12, 20, 27])
```

a) Viết lệnh lấy hàng giá cao nhất, giữ id và gia; tính mean và median ban đầu.

b) Chỉ để khảo sát độ nhạy, tạo bảng sau bằng cách drop hàng đó theo nhãn index; giữ df nguyên. Tính mean và median của sau.

c) Giải thích ảnh hưởng của giá 860 lên mean so với median. Có đủ căn cứ kết luận giá 860 hoặc giá thấp nhất 20 là sai không? Cần giữ thông tin gì để kiểm chứng nguồn?

B4-029

Yêu cầu: thử đổi giá b thành 50 trong thu, giữ giá gốc. Đã chạy hết:

```
goc = pd.DataFrame({"loai": ["A", "B", "A"], "gia": [60., 80., 100.]},
                   index=["a", "b", "c"])
truoc = goc.copy()
thu = goc
thu.loc["b", "gia"] = 50.
print(goc["gia"].tolist())
```

a) Dòng cuối in gì? Vì sao CoW không bảo vệ goc trong tình huống này?

b) Từ trạng thái đã chạy, dùng truoc để khôi phục goc, rồi tạo thu riêng có giá b=50. Viết vài dòng.

c) Nêu phép kiểm cụ thể cho cả hai bảng; chỉ thêm copy sau khi goc đã bị sửa có tự khôi phục giá 80 không?

B4-030

```
df = pd.DataFrame({"loai": ["Studio", "PR", "Studio", "PR", "PR"],
                   "gia": [90., 60., 95., 55., np.nan]},
                  index=["S1", "S2", "S3", "S4", "S5"])
m = (((df["loai"] == "Studio") and (df["gia"] <= 90))
     or ((df["loai"] == "PR") and (df["gia"] < 60)))
```

a) Vì sao dòng tạo m báo lỗi?

b) Sửa biểu thức để chọn Studio giá không quá 90 hoặc PR giá dưới 60.

c) Nêu nhãn được giữ; giải thích riêng các hàng giá 60 và NaN.

Đợt 4 — B4-031 đến B4-040

7 câu đầu trắc nghiệm; 3 câu cuối tự luận.

B4-031

Mỗi hàng là một chỗ ở; id là mã tra cứu, gia tính bằng USD/đêm.

```
df = pd.DataFrame({"id": [10, 20, 90], "gia": [50., 60., 100.]})
print(df["id"].mean(), df["gia"].mean())
```

Dòng cuối in `40.0 70.0` . Nhận định nào đúng?

A. Cả 40 và 70 đều là trung bình giá, đơn vị USD/đêm.
B. 40 là mã chỗ ở điển hình nên có thể dùng làm mức giá đại diện.
C. 40 là trung bình mã về toán học, không phải mức giá; 70 là trung bình giá.
D. Code phải báo TypeError vì không thể tính mean trên cột id kiểu số nguyên.

B4-032

```
s = pd.Series([10.2, 20.2, np.nan, 30.2, 40.2], dtype="float64")
d = s.describe().round(0)
print(d.loc[["count", "25%", "50%", "75%"]].tolist(), s.iloc[0])
```

Dòng cuối in gì?

A. `[5.0, 18.0, 25.0, 33.0] 10.2`
B. `[4.0, 18.0, 25.0, 33.0] 10.0`
C. `[4.0, 18.0, 25.0, 33.0] 10.2`
D. `[4.0, 25.0, 50.0, 75.0] 10.2`

B4-033

```
s = pd.Series([10., 20., 30., np.nan, np.nan, 90.], dtype="float64")
p = (s < 50).mean()
q = (s[s.notna()] < 50).mean()
print(p, q, (q - p) * 100)
```

Diễn giải nào đúng về ba số được in?

A. p=0,5%; q=0,75%; chênh lệch 25 điểm phần trăm.
B. p=50%; q=75%; chênh lệch 25 điểm phần trăm.
C. p=50%; q=75%; q tăng tương đối 25% so với p.
D. p=75%; q=50%; chênh lệch −25 điểm phần trăm.

B4-034

```
df = pd.DataFrame({"id": ["T1", "T2", "T3", "T4", "T5"],
                   "gia": [30., 150., 80., 120., 60.]})
r = df.nlargest(2, "gia")[["gia", "id"]]
print(r["id"].tolist(), r.columns.tolist())
```

Dòng cuối in gì?

A. `['T2', 'T4'] ['gia', 'id']`
B. `['T1', 'T5'] ['gia', 'id']`

C. `['T4', 'T2'] ['gia', 'id']`

D. `['T2', 'T4'] ['id', 'gia']`

B4-035

```
df = pd.DataFrame({"gia": [10., 20., 30.], "phi": [1., 2., 3.]},
                  dtype="float64")
b = df.copy()
a = b
a.loc[0, "gia"] = 99.
print(df.loc[0, "gia"], b.loc[0, "gia"])
```

Dòng cuối in gì?

A. `10.0 99.0`

B. `99.0 99.0`

C. `10.0 10.0`

D. `99.0 10.0`

B4-036

gia là USD/đêm; phi chỉ thu một lần. Cần báo giá ba đêm theo giá mới.

```
df = pd.DataFrame({"gia": [50., 70., 90.], "phi": [20., 10., 0.]},
                  index=["a", "b", "c"], dtype="float64")
df["tong"] = (df["gia"] + df["phi"]) * 3
df.loc["a", "gia"] = 40.
print(df.loc["a", ["gia", "tong"]].tolist())
```

Phương án nào nêu đúng (kết quả in; tổng đúng cho a theo yêu cầu)?

A. `([40.0, 140.0]; 140.0)`

B. `([40.0, 180.0]; 140.0)`

C. `([40.0, 170.0]; 140.0)`

D. `([40.0, 210.0]; 140.0)`

B4-037

```
df = pd.DataFrame({"gia": [10., 20., 30.], "phi": [0., 2., 4.]},
                  index=["A", "B", "C"], dtype="float64")
gui = df[["gia"]]
```

Người nhận dùng read_csv thông thường và cần đúng hai cột ma, gia, với ma là A,B,C. Lệnh nào đáp ứng đủ?

A. `gui.to_csv("x.csv", index=False, index_label="ma")`

B. `gui.to_csv("x.csv", index=True, index_label="id")`

C. `gui.to_csv("x.csv", index=False, columns=["ma", "gia"])`

D. `gui.to_csv("x.csv", index_label="ma")`

B4-038

Giá 0 là miễn phí hợp lệ; hai NaN là chưa biết giá. Cần trung bình các giá đã biết.

```
s = pd.Series([0., 20., np.nan, 40., np.nan], dtype="float64")
tb = s.sum() / len(s)
print(tb)
```

a) Code in gì và sai ở mẫu số nào?

b) Sửa một dòng để đúng yêu cầu; nêu số giá được dùng và kết quả.

c) Một bạn đề xuất thay hai ô thiếu bằng 0 rồi giữ nguyên công thức. Cách đó có đáp ứng ý nghĩa yêu cầu không? Vì sao giá 0 vốn có vẫn phải được giữ?

B4-039

gia là USD/đêm, phi thu một lần. Cần giảm 10% giá B trên bản sao, giữ nguồn và báo giá ba đêm.

```
nguon = pd.DataFrame({"gia": [90., 70., 50.], "phi": [0., 40., 120.]},
                     index=["A", "B", "C"], dtype="float64")
bao = nguon.copy()
bao["tong"] = (bao["gia"] + bao["phi"]) * 3
bao.loc["B", "gia"] *= 0.9
print(bao.loc["B", ["gia", "tong"]].tolist())
```

a) Dòng cuối in gì? Chỉ ra hai vấn đề của cột tong.

b) Viết lại phần tạo bao từ nguon để đúng yêu cầu, nêu tong của cả ba hàng và phép kiểm nguồn.

c) Với giá sau giảm và phí vẫn thu một lần, từ bao nhiêu đêm nguyên dương C rẻ hơn cả A và B? Kiểm tại mốc và ngay trước mốc.

B4-040

```
df = pd.DataFrame({"gia": [100., 20., 40., 120., 60.],
                   "phi": [0., 0., 0., 0., 0.]},
                  index=[0, 1, 3, 4, 6], dtype="float64")
r = df[df["gia"] < 100]
print(r.loc[1:3, "gia"].sum())
```

Yêu cầu thực sự là cộng giá của hàng thứ hai và thứ ba trong r.

a) Dòng cuối in gì? Nêu các nhãn còn lại và lý do khác yêu cầu.

b) Sửa bằng iloc với lát cắt; cho biết tổng đúng.

c) Trong hai cách viết, nhãn cuối của loc và stop của iloc được xử lý thế nào?

Đợt 5 — B4-041 đến B4-050

6 câu đầu trắc nghiệm; 4 câu cuối tự luận.

B4-041

Mỗi hàng trong snapshot giả lập là một chỗ ở.

```
df = pd.DataFrame({"quan": ["A", "A", "A", "B", "B", "B", "B"],
                   "loai": ["NC", "PR", "NC", "NC", "PR", "NC", "PR"],
                   "gia": [10., 20., np.nan, 40., 50., 60., 70.]})
print(df.shape)
print(df.sample(2, random_state=1))
df.info()
print(df["gia"].describe())
print(df["loai"].value_counts())
```

Trích kết quả năm bước: shape (7,3); mẫu gồm hàng 6 (B,PR,70) và hàng 2 (A,NC,NaN); gia có 6 non-null; describe có count 6 và trung vị45; value_counts cho NC4, PR3.

Nhận định nào đúng khi lập hồ sơ cho snapshot này?

A. B chiếm 1/2 số chỗ ở vì mẫu có 1 hàng B trên2; PR chiếm 3/7; thiếu 1 giá.
B. B chiếm 4/7; PR chiếm 3/7; thiếu 1 giá; trung vị các giá biết là45; mẫu chỉ để xem nhanh.
C. B chiếm 4/6 vì chỉ có 6 giá biết; PR chiếm 3/7; thiếu 1 giá; trung vị45.
D. B chiếm 4/7; PR chiếm 4/7; thiếu 1 giá; trung vị các giá biết là45.

B4-042

```
s = pd.Series([20., 30., 40., 910.], dtype="float64")
print(s.mean(), s.median())
```

Phương án nào đúng cả kết quả và nhận xét?

A. `35.0 250.0` ; giá 910 kéo mean cao hơn median.
B. `250.0 40.0` ; median luôn là một giá có sẵn trong bảng.
C. `250.0 35.0` ; vì mean lớn hơn median nên giá 910 chắc chắn sai.
D. `250.0 35.0` ; giá 910 kéo mean lên, nhưng chưa đủ căn cứ nói giá đó sai.

B4-043

```
df = pd.DataFrame({"id": ["A", "B", "C", "D", "E"],
                   "loai": ["NC", "PR", "Shared", "NC", "PR"],
                   "gia": [95., 40., 10., 60., 50.]})
m = df["loai"].isin(["NC", "PR"])
r = df.loc[m, ["id", "gia"]].nsmallest(2, "gia")
print(r["id"].tolist())
```

Dòng cuối in gì?

A. `['B', 'E']`
B. `['C', 'B']`
C. `['A', 'D']`
D. `['A', 'B']`

B4-044

```
df = pd.DataFrame({"gia": [15., 25., 35.], "phi": [0., 0., 0.]},
                  index=[2, 0, 1], dtype="float64")
print(df.loc[0, "gia"], df.iloc[0, 0])
```

Dòng cuối in gì?

A. `15.0 15.0`

B. `25.0 15.0`

C. `25.0 25.0`

D. `15.0 25.0`

B4-045

```
df = pd.DataFrame({"gia": [10., 20., 30.], "phi": [0., 1., 2.]},
                  dtype="float64")
b = df.copy()
s = b["gia"]
s.iloc[1] = 99.
b.loc[0, "gia"] = 5.
print(df["gia"].tolist(), b["gia"].tolist(), s.tolist())
```

Dòng cuối in gì?

A. `[10.0,20.0,30.0] [5.0,20.0,30.0] [10.0,99.0,30.0]`

B. `[5.0,20.0,30.0] [5.0,20.0,30.0] [10.0,99.0,30.0]`

C. `[10.0,20.0,30.0] [5.0,99.0,30.0] [10.0,99.0,30.0]`

D. `[10.0,20.0,30.0] [10.0,20.0,30.0] [10.0,99.0,30.0]`

B4-046

```
df = pd.DataFrame({"id": ["R1", "R2", "R3"], "gia": [10., np.nan, 90.]})
df.to_csv("bang_046.csv", index=False)
lai = pd.read_csv("bang_046.csv")
print(lai.shape, lai.columns.tolist(), lai["gia"].isna().sum(),
      lai["gia"].sum() == df["gia"].sum())
```

Phương án nào đúng cả kết quả và giới hạn của phép kiểm?

A. `(3,2) ['id','gia'] 1 True` ; đã chứng minh mọi giá nguồn đều đúng thực tế.

B. `(3,2) ['id','gia'] 0 True` ; CSV đã tự sửa giá thiếu thành 0.

C. `(3,2) ['id','gia'] 1 True` ; các kiểm tra này khớp, chưa chứng minh giá nguồn đúng.

D. `(3,2) ['id','gia'] 1 False` ; NaN làm tổng đọc lại khác tổng ban đầu.

B4-047

id là duy nhất ở mỗi bảng; không thiếu giá. Thứ tự hàng sau khi đọc không phải yêu cầu cần giữ.

```
goc = pd.DataFrame({"id": ["A", "B", "C"], "gia": [15., 35., 50.]})
gui = pd.DataFrame({"id": ["A", "B", "C"], "gia": [50., 15., 35.]})
gui.to_csv("bang_047.csv", index=False)
lai = pd.read_csv("bang_047.csv")
ok = (lai.shape == goc.shape) and (lai["gia"].sum() == goc["gia"].sum())
print(ok)
```

a) Dòng cuối in gì? Vì sao kết quả đó chưa kiểm được yêu cầu giữ đúng cặp id–giá?

b) Viết vài dòng kiểm đúng quan hệ id–giá so với goc, chấp nhận thứ tự hàng khác nhau. Không dùng merge hoặc khớp nhãn giữa hai Series.

c) Nếu phép kiểm ở b) cho True, có đủ kết luận giá trong goc đúng thực tế không? Nêu thêm việc cần làm khi nghi giá bất thường.

B4-048

Cần bao gồm đủ bốn chỗ ở, giảm 50% giá NC, giữ giá PR, tính hai đêm theo giá mới và giữ nguon nguyên.

```
nguon = pd.DataFrame({"loai": ["NC", "PR", "NC", "PR"],
                      "gia": [100., 80., 60., 40.]}, index=["w", "x", "y", "z"])
bao = nguon.copy()
bao["hai"] = bao["gia"] * 2
m = bao["loai"] == "NC"
bao = bao.loc[m].copy()
bao.loc[:, "gia"] *= 0.5
```

a) Nêu shape, giá và tiền hai đêm còn trong bao. Giá NC đã giảm đúng có nghĩa đã đủ yêu cầu không?

b) Viết lại từ nguon để đáp ứng đủ yêu cầu.

c) Nêu kiểm tra cụ thể cho số hàng, giá PR, cột hai và bảng nguồn.

B4-049

```
s = pd.Series(["NC", "PR", np.nan, "NC", "Shared", np.nan])
```

a) Viết lệnh và nêu kết quả value_counts() và value_counts(normalize=True). Không yêu cầu thứ tự giữa các nhãn bằng số đếm.

b) Mẫu số của kết quả normalize là bao nhiêu? Chia mọi số đếm cho6 có tái hiện kết quả đó không?

c) Viết lệnh đếm nhãn thiếu và nêu số thiếu cùng tỷ lệ thiếu trên toàn Series.

B4-050

```
df = pd.DataFrame({"id": ["P1", "P2", "P3", "P4", "P5", "P6"],
                   "khu": ["Đông", "Đông", "Tây", "Đông", "Tây", "Đông"]})
```

Một lần đo trên môi trường kiểm chứng cho số liệu sau, dùng memory_usage(deep=True) cùng index trước/sau chuyển astype("category"):

Cột	Trước chuyển (byte)	Sau chuyển (byte)
id	438	444
khu	564	292

a) Với khu, category biểu diễn sáu hàng bằng nhãn và mã tham chiếu như thế nào?

b) Diễn giải chiều thay đổi bộ nhớ của hai cột trong lần đo này. Vì sao cần deep=True khi ước tính dữ liệu chuỗi?

c) Cột nào phù hợp hơn về bản chất dữ liệu? Có thể kết luận chuyển mọi cột sang category luôn tiết kiệm không? Không cần nhớ hay tính số byte tuyệt đối.

Đợt 6 — B4-051 đến B4-060

6 câu đầu trắc nghiệm; 4 câu cuối tự luận.

B4-051

```
du_lieu = {"id": [4, 9, 8], "gia": [30., 60., 90.]}
nhan = ["K1", "K2", "K3"]
```

Cần DataFrame có nhãn hàng K1,K2,K3, hai cột id,gia; id là mã tra cứu. Phương án nào đúng cả cách tạo và nhận xét?

A. `pd.DataFrame(du_lieu)` ; bảng tự có nhãn K1,K2,K3.

B. `pd.DataFrame(du_lieu, index=nhan)` ; shape(3,2), index ngoài cột dữ liệu, id là khóa.

C. `pd.DataFrame({"ma_hang": nhan, "id": du_lieu["id"], "gia": du_lieu["gia"]})` ; shape(3,2), nhãn hàng là nhan.

D. `pd.DataFrame(du_lieu, index=nhan)` ; trung bình id=7 là mức giá điển hình.

B4-052

```
s = pd.Series([20., 50., 80., np.nan], dtype="float64")
p = (s < 50).mean()
q = (s >= 50).mean()
print(p, q, p + q)
```

Nhận định nào đúng?

A. In `0.25 0.75 1.0` ; mọi False của <50 đều là True của >=50.

B. In `0.25 0.5 0.75` ; NaN không thỏa cả hai phép so sánh.

C. In `0.3333333333333333 0.6666666666666666 1.0` ; mean tự loại hàng NaN trước khi so sánh.

D. In `0.5 0.5 1.0` ; giá 50 được tính vào cả hai nhóm.

B4-053

```
df = pd.DataFrame({"gia": [10., 20., 30.], "phi": [1., 2., 3.]},
                  index=["a", "b", "c"], dtype="float64")
a = df.loc["b", ["phi", "gia"]]
b = df.loc[["b"], ["phi", "gia"]]
print(type(a).__name__, a.shape, type(b).__name__, b.shape)
```

Dòng cuối in gì?

A. `DataFrame (1, 2) DataFrame (1, 2)`

B. `Series (1,) DataFrame (1, 2)`

C. `Series (2,) DataFrame (2, 1)`

D. `Series (2,) DataFrame (1, 2)`

B4-054

```
df = pd.DataFrame({"gia": [10., 20., 30.], "phi": [0., 1., 2.]},
                  index=["a", "b", "c"], dtype="float64")
df["hai"] = df["gia"] * 2
a = df
a["gia"]["b"] = 5.
a.loc["a", "gia"] = 8.
print(df["gia"].tolist(), df["hai"].tolist())
```

Theo quy ước cảnh báo không dừng chương trình, dòng cuối in gì?

A. `[8.0,5.0,30.0] [16.0,10.0,60.0]`

B. `[8.0,5.0,30.0] [20.0,40.0,60.0]`

C. `[10.0,20.0,30.0] [20.0,40.0,60.0]`

D. `[8.0,20.0,30.0] [20.0,40.0,60.0]`

B4-055

```
df = pd.DataFrame({"gia": [15., np.nan, 45.], "loai": ["A", "B", "A"]},
                  index=["S3", "S1", "S2"])
df.index.name = "id"
df.reset_index().to_csv("bang_055.csv", index=False)
lai = pd.read_csv("bang_055.csv")
print(lai.shape, lai.columns.tolist(), lai["gia"].isna().sum(), lai["gia"].sum())
```

Dòng cuối in gì?

A. `(3,2) ['gia','loai'] 1 60.0`

B. `(3,4) ['Unnamed: 0','id','gia','loai'] 1 60.0`

C. `(3,3) ['id','gia','loai'] 1 60.0`

D. `(3,3) ['id','gia','loai'] 0 60.0`

B4-056

```
s = pd.Series(["A", "B", np.nan, "A", "C", "B"])
v = s.value_counts(normalize=True)
print(v["A"], s.nunique(), s.count())
```

Dòng cuối in gì?

A. `0.4 3 5`

B. `0.3333333333333333 3 6`

C. `0.4 4 5`

D. `2 3 5`

B4-057

Mỗi hàng là một chỗ ở trong snapshot giả lập, gia tính bằng USD/đêm.

```
df = pd.DataFrame({"id": ["P1", "P2", "P3", "P4", "P5", "P6"],
                   "quan": ["A", "A", "B", "B", "B", "C"],
                   "gia": [20., 40., 50., np.nan, 70., 100.]})
TEN_QUAN = "B"
```

a) Viết vài dòng tính số chỗ ở của quận, tỷ trọng số hàng của quận trong toàn bảng, trung vị giá quận và toàn bảng. Code chỉ cần đổi TEN_QUAN để dùng cho quận khác trong bảng.

b) Cho kết quả với B và tính giá trung vị B cao hơn trung vị toàn bảng bao nhiêu phần trăm.

c) Nêu mẫu số dùng cho tỷ trọng số chỗ ở và số giá tham gia trung vị B. Có được gọi tỷ trọng đó là tỷ lệ lượt đặt phòng hoặc suy rộng ngay thành thị phần toàn thành phố thực tế không?

B4-058

```
df = pd.DataFrame({"id": ["U1", "U2", "U3", "U4", "U5", "U6", "U7"],
                   "loai": ["NC", "PR", "Shared", "NC", "PR", "NC", "Shared"],
                   "gia": [100., 40., 5., 80., 60., 20., 10.]})
```

a) Dùng isin và nsmallest để lấy ba chỗ ở rẻ nhất trong hai loại NC/PR; giữ đúng cột id,gia theo thứ tự đó và nhãn gốc.

b) Nêu mã, giá và nhãn hàng theo thứ tự kết quả.

c) Vì sao phải giữ mã khi kiểm tra giá thấp nhất? Giá thấp nhất được chọn có chắc là lỗi không?

B4-059

Cần giảm 50% giá loại A trong bao, giữ giá loại B, tính tiền ba đêm theo giá mới và giữ goc nguyên.

```
goc = pd.DataFrame({"loai": ["A", "B", "A"], "gia": [40., 60., 80.]},
                   index=["p", "q", "r"])
bao = goc.copy()
bao["ba_dem"] = bao["gia"] * 3
tam = bao[["loai", "gia"]].copy()
m = tam["loai"] == "A"
tam.loc[m, "gia"] *= 0.5
tam["ba_dem"] = tam["gia"] * 3
print(bao["ba_dem"].sum(), tam["ba_dem"].sum())
```

a) Dòng cuối in gì? Vì sao sửa đúng tam và tính lại ba_dem trong tam chưa hoàn thành yêu cầu?

b) Từ trạng thái này, viết vài dòng sửa đúng bao mà vẫn giữ goc nguyên.

c) Nếu chỉ sửa giá trong bao rồi dừng, tổng ba_dem của bao đã đúng chưa? Nêu kết quả cuối và kiểm hàng loại B, bảng nguồn.

B4-060

```
nguon = pd.DataFrame({"gia": [15., 45., 30.], "phi": [0., 5., 10.]},
                     index=["R7", "R2", "R9"], dtype="float64")
nguon.index.name = "ma"
gui = nguon[["gia"]].sort_values("gia")
gui.to_csv("bang_060.csv", index=False)
lai = pd.read_csv("bang_060.csv")
```

a) Người nhận chỉ có bang_060.csv có xác định được giá nào thuộc mã nào không? Vì sao không thể lấy lại mã chỉ từ thứ tự hàng đã xuất?

b) Khi nguon/gui vẫn còn, sửa cách xuất để giữ đúng hai cột ma,gia; viết lệnh đọc lại. Chấp nhận giữ index hoặc chuyển thành cột.

c) Nêu mã–giá theo thứ tự file đúng và kiểm tra tối thiểu gồm shape, columns, thiếu, tổng, cặp mã–giá. Nếu đều khớp, đã chứng minh giá nguồn đúng thực tế chưa?